

Testes Unitários com Jest

unit/taskRepository.test.js

```
const repository = require("../repositories/taskRepository");

describe("Task Repository", () => {
  beforeEach(() => {
    // Limpar o array de tarefas antes de cada teste
    repository.findAll().length = 0;
  });

  test("deve salvar uma tarefa", () => {
    // Arrange (preparação)
    const task = { id: 1, title: "Tarefa 1" };

    // Act (ação)
    repository.save(task);
    const tasks = repository.findAll();

    // Assert (verificações)
    expect(tasks).toContain(task);
    expect(tasks).toHaveLength(1);
  });

  test("deve salvar múltiplas tarefas", () => {
    const task1 = { id: 1, title: "Tarefa 1" };
    const task2 = { id: 2, title: "Tarefa 2" };
    const task3 = { id: 3, title: "Tarefa 3" };

    repository.save(task1);
    repository.save(task2);
    repository.save(task3);

    const tasks = repository.findAll();

    expect(tasks).toHaveLength(3);
    expect(tasks).toEqual([task1, task2, task3]);
  });

  test("deve retornar array vazio quando nenhuma tarefa foi salva", () => {
    const tasks = repository.findAll();

    expect(tasks).toEqual([]);
    expect(Array.isArray(tasks)).toBe(true);
  });

  test("deve manter ordem de inserção das tarefas", () => {
    const task1 = { id: 1, title: "Primeira" };
    const task2 = { id: 2, title: "Segunda" };
    const task3 = { id: 3, title: "Terceira" };

    repository.save(task1);
    repository.save(task2);
    repository.save(task3);

    const tasks = repository.findAll();

    expect(tasks[0]).toBe(task1);
    expect(tasks[1]).toBe(task2);
    expect(tasks[2]).toBe(task3);
  });

  test("deve retornar a mesma referência de array", () => {
```

```

    const task = { id: 1, title: "Tarefa" };
    repository.save(task);

    const tasks1 = repository.findAll();
    const tasks2 = repository.findAll();

    expect(tasks1).toBe(tasks2);
  });
});

```

unit/taskService.test.js

```

jest.mock("../repositories/taskRepository", () => ({
  save: jest.fn(),
  findAll: jest.fn(),
}));

const repository = require("../repositories/taskRepository");
const taskService = require("../services/taskService");

describe("Task Service", () => {
  beforeEach(() => {
    jest.clearAllMocks();
  });

  test("deve criar tarefa", () => {
    const tarefa = taskService.addTask("Estudar");

    expect(tarefa.title).toBe("Estudar");

    expect(repository.save).toHaveBeenCalledTimes(1);

    expect(repository.save).toHaveBeenCalledWith(tarefa);
  });

  test("deve lançar erro sem título", () => {
    expect(() => {
      taskService.addTask("");
    }).toThrow("Título obrigatório");

    expect(repository.save).not.toHaveBeenCalled();
  });

  test("deve listar tarefas", () => {
    repository.findAll.mockReturnValue([
      { id: 1, title: "Tarefa 1" },
      { id: 2, title: "Tarefa 2" },
    ]);

    const tarefas = taskService.getTasks();

    expect(tarefas).toHaveLength(2);

    expect(repository.findAll).toHaveBeenCalledTimes(1);
  });
});

```

unit/taskService-advanced.test.js

```
jest.mock("../repositories/taskRepository", () => ({
  save: jest.fn(),
  findAll: jest.fn(),
}));

const repository = require("../repositories/taskRepository");
const taskService = require("../services/taskService");

describe("Task Service - Testes Avançados", () => {
  beforeEach(() => {
    jest.clearAllMocks();
  });

  describe("addTask - Validações de Tipos", () => {
    test("deve rejeitar título null", () => {
      expect(() => {
        taskService.addTask(null);
      }).toThrow("Título obrigatório");

      expect(repository.save).not.toHaveBeenCalled();
    });

    test("deve rejeitar título undefined", () => {
      expect(() => {
        taskService.addTask(undefined);
      }).toThrow("Título obrigatório");

      expect(repository.save).not.toHaveBeenCalled();
    });

    test("deve rejeitar título como array", () => {
      expect(() => {
        taskService.addTask([]);
      }).toThrow("Título deve ser uma string");

      expect(repository.save).not.toHaveBeenCalled();
    });

    test("deve rejeitar título como objeto", () => {
      expect(() => {
        taskService.addTask({});
      }).toThrow("Título deve ser uma string");

      expect(repository.save).not.toHaveBeenCalled();
    });

    test("deve rejeitar título como booleano", () => {
      expect(() => {
        taskService.addTask(true);
      }).toThrow("Título deve ser uma string");

      expect(repository.save).not.toHaveBeenCalled();
    });
  });

  describe("addTask - Geração de ID", () => {
    test("deve gerar ID do tipo number", () => {
      const task = taskService.addTask("Tarefa com ID");

      expect(typeof task.id).toBe("number");
    });

    test("deve usar timestamp como ID", () => {
      const beforeTime = Date.now();
      const task = taskService.addTask("Tarefa");
      const afterTime = Date.now();
    });
  });
});
```

```

        expect(task.id).toBeGreaterThanOrEqual(beforeTime);
        expect(task.id).toBeLessThanOrEqual(afterTime);
    });
});

describe("addTask - Preservação de Dados", () => {
    test("deve preservar título com caracteres especiais", () => {
        const specialTitle = "Tarefa @#%&*&*(!";
        const task = taskService.addTask(specialTitle);

        expect(task.title).toBe(specialTitle);
    });

    test("deve preservar título com espaços normais", () => {
        const titleWithSpaces = "Comprar pão e leite";
        const task = taskService.addTask(titleWithSpaces);

        expect(task.title).toBe(titleWithSpaces);
    });

    test("deve preservar maiúsculas e minúsculas", () => {
        const mixedCaseTitle = "TaReFA Com MaÍúScULaS";
        const task = taskService.addTask(mixedCaseTitle);

        expect(task.title).toBe(mixedCaseTitle);
    });

    test("deve preservar acentuação", () => {
        const accentedTitle = "Tarefa com acento: áéíóú";
        const task = taskService.addTask(accentedTitle);

        expect(task.title).toBe(accentedTitle);
    });

    test("deve aceitar título longo", () => {
        const longTitle = "A".repeat(1000);
        const task = taskService.addTask(longTitle);

        expect(task.title).toBe(longTitle);
        expect(task.title.length).toBe(1000);
    });
});

describe("addTask - Estrutura do Retorno", () => {
    test("deve retornar objeto com propriedades id e title", () => {
        const task = taskService.addTask("Nova tarefa");

        expect(task).toHaveProperty("id");
        expect(task).toHaveProperty("title");
    });

    test("deve retornar apenas id e title", () => {
        const task = taskService.addTask("Nova tarefa");
        const keys = Object.keys(task);

        expect(keys).toEqual(["id", "title"]);
    });

    test("deve chamar repository.save com o objeto correto", () => {
        const task = taskService.addTask("Test task");

        expect(repository.save).toHaveBeenCalledWith({
            id: task.id,
            title: "Test task",
        });
    });
});

describe("getTasks - Funcionamento", () => {

```

```

test("deve retornar lista vazia quando nenhuma tarefa existe", () => {
  repository.findAll.mockReturnValue([]);

  const tasks = taskService.getTasks();

  expect(tasks).toEqual([]);
  expect(tasks).toHaveLength(0);
});

test("deve retornar todas as tarefas", () => {
  const mockTasks = [
    { id: 1, title: "Tarefa 1" },
    { id: 2, title: "Tarefa 2" },
    { id: 3, title: "Tarefa 3" },
  ];

  repository.findAll.mockReturnValue(mockTasks);

  const tasks = taskService.getTasks();

  expect(tasks).toEqual(mockTasks);
  expect(tasks).toHaveLength(3);
});

test("deve chamar repository.findAll", () => {
  repository.findAll.mockReturnValue([]);

  taskService.getTasks();

  expect(repository.findAll).toHaveBeenCalledTimes(1);
  expect(repository.findAll).toHaveBeenCalledWith();
});

test("deve retornar a mesma referência do repository", () => {
  const mockTasks = [{ id: 1, title: "Tarefa" }];

  repository.findAll.mockReturnValue(mockTasks);

  const tasks = taskService.getTasks();

  expect(tasks).toBe(mockTasks);
});

describe("Integração de Métodos", () => {
  test("deve adicionar e recuperar tarefa criada", () => {
    repository.findAll.mockReturnValue([]);

    // Adiciona tarefa
    const addedTask = taskService.addTask("Tarefa integrada");

    // Simula que o repository agora retorna a tarefa
    repository.findAll.mockReturnValue([addedTask]);

    const tasks = taskService.getTasks();

    expect(tasks).toContain(addedTask);
    expect(tasks[0].title).toBe("Tarefa integrada");
  });

  test("deve adicionar múltiplas tarefas e recuperar todas", () => {
    repository.findAll.mockReturnValue([]);

    const task1 = taskService.addTask("Tarefa 1");
    const task2 = taskService.addTask("Tarefa 2");

    repository.findAll.mockReturnValue([task1, task2]);

    const tasks = taskService.getTasks();
  });
}

```

```
    expect(tasks).toHaveLength(2);
    expect(tasks).toContain(task1);
    expect(tasks).toContain(task2);
  });
});
});
```